

Banned syscalls for Part A:
exec* (any syscall that starts with exec)
At: Shell Prompt [3] Your shell should show a prompt so the user knows it's ready for input...

PASTED

Give me the skeleton code and I will fill the rest

Ran a command, read a file

Skeleton for Part A — six headers, seven sources, and a Makefile.

Already written for you: all headers (the `Token / TokenList` and `Job / Command / JobList` structures, plus the constants), `a_command.c` in full, and inside the lexer the helpers `is_space`, `is_special`, `add_token`, `tokenlist_free`, and the complete `LEX_START` state so you can see the pattern before writing the rest.

Yours to fill:

File	Blank
<code>a_shell.c</code>	<code>shell_init</code> — record the launch directory
<code>a_prompt.c</code>	<code>shorten_path</code> , and assembling the prompt string
<code>a_input.c</code>	the body of the read loop
<code>a_lexer.c</code>	five states: <code>LEX_WORD</code> , <code>LEX_ESCAPE</code> , <code>LEX_DQUOTE</code> , <code>LEX_DQ_ESCAPE</code> , <code>LEX_SQUOTE</code>
<code>a_parser.c</code>	<code>PARSE_ARGS</code> , <code>PARSE_NEED_WORD</code> , and the end-of-line validity check
<code>a_main.c</code>	the lex → parse section of the loop



Do them in that order — A1 and A2 are 5 marks and quick, then the 15-mark lexer and parser

Write a message...



Sonnet 4.6 Low



Copy



```
1 CC := gcc
2 CFLAGS := -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 \
3           -Wall -Wextra -Werror -Wno-unused-parameter -fno-asm \
4           -Iinclude
5 LDFLAGS :=
6
7 SRC_DIR := src
8 BUILD_DIR := build
9 TARGET := shell.out
10
11 SRCS := $(wildcard $(SRC_DIR)/*.c)
12 OBJS := $(patsubst $(SRC_DIR)/%.c,$(BUILD_DIR)/%.o,$(SRCS))
13
14 .PHONY: all clean
15
16 all: $(TARGET)
17
18 $(TARGET): $(OBJS)
19     $(CC) $(OBJS) -o $@ $(LDFLAGS)
20
21 $(BUILD_DIR)/%.o: $(SRC_DIR)/%.c | $(BUILD_DIR)
22     $(CC) $(CFLAGS) -c $< -o $@
23
24 $(BUILD_DIR):
25     mkdir -p $(BUILD_DIR)
26
27 clean:
28     rm -rf $(BUILD_DIR) $(TARGET)
```



Do them in that order — A1 and A2 are 5 marks and quick, then the 15-mark lexer and parser once the loop around them already works. Call `joblist_dump(&jobs)` from `main` while building; seeing your parse tree printed makes the parser far easier to debug than guessing.

Two traps the spec examples will catch:

In `LEX_WORD`, when a space or special character ends the word, don't advance `pos`. Emit the token, return to `LEX_START`, and let that state read the character — otherwise `echo a|sort` silently swallows the pipe.

For the end-of-line check in the parser, `echo hi &` must be valid while `echo hi ;` must not. Look at the grammar: `BG → ε` exists, `CMD → ε` doesn't. One state can legally end the line, the other can't.

Build it with the included Makefile. If your compiler rejects `-std=c23`, it's older than gcc 14 — use `-std=c2x` locally, but keep `c23` in what you submit.



A lexer
c

[Download](#)

A parser
c

[Download](#)

A prompt
c

[Download](#)

A input
c

[Download](#)

A shell
c

[Download](#)

A main
c

[Download](#)

Write a message...



Sonnet 4.6 Low ▾



```
1 CC := gcc
2 CFLAGS := -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 \
3         -Wall -Wextra -Werror -Wno-unused-parameter -fno-asm \
4         -Iinclude
5 LDFLAGS :=
6
7 SRC_DIR := src
8 BUILD_DIR := build
9 TARGET := shell.out
10
11 SRCS := $(wildcard $(SRC_DIR)/*.c)
12 OBJS := $(patsubst $(SRC_DIR)/%.c,$(BUILD_DIR)/%.o,$(SRCS))
13
14 .PHONY: all clean
15
16 all: $(TARGET)
17
18 $(TARGET): $(OBJS)
19     $(CC) $(OBJS) -o $@ $(LDFLAGS)
20
21 $(BUILD_DIR)/%.o: $(SRC_DIR)/%.c | $(BUILD_DIR)
22     $(CC) $(CFLAGS) -c $< -o $@
23
24 $(BUILD_DIR):
25     mkdir -p $(BUILD_DIR)
26
27 clean:
28     rm -rf $(BUILD_DIR) $(TARGET)
```

-  A shell
C [Download](#)
-  A main
C [Download](#)
-  A command
C [Download](#)
-  A lexer
H [Download](#)
-  A command
H [Download](#)
-  A parser
H [Download](#)
-  A shell
H [Download](#)
-  A prompt
H [Download](#)
-  A input
H [Download](#)
-  Makefile [Download](#)

 Download all

Write a message...



Sonnet 4.6 Low



Copy



```
1 CC := gcc
2 CFLAGS := -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 \
3           -Wall -Wextra -Werror -Wno-unused-parameter -fno-asm \
4           -Iinclude
5 LDFLAGS :=
6
7 SRC_DIR := src
8 BUILD_DIR := build
9 TARGET := shell.out
10
11 SRCS := $(wildcard $(SRC_DIR)/*.c)
12 OBJ := $(patsubst $(SRC_DIR)/%.c,$(BUILD_DIR)/%.o,$(SRCS))
13
14 .PHONY: all clean
15
16 all: $(TARGET)
17
18 $(TARGET): $(OBJ)
19     $(CC) $(OBJ) -o $@ $(LDFLAGS)
20
21 $(BUILD_DIR)/%.o: $(SRC_DIR)/%.c | $(BUILD_DIR)
22     $(CC) $(CFLAGS) -c $< -o $@
23
24 $(BUILD_DIR):
25     mkdir -p $(BUILD_DIR)
26
27 clean:
28     rm -rf $(BUILD_DIR) $(TARGET)
```

Banned syscalls for Part B:
exec* (any syscall that starts with exec) B: hop [10]
Syntax: hop ((- | . | - | na
me)*)? Purpose: The hop
command changes the s...

PASTED

Now we will do part B, same thing, give me the skeleton and I will complete

3 minutes ago

Ran 3 commands, read a file

Part B skeleton — two headers, two sources. Drop them into your `include/` and `src/`, and have `main` route builtin names to `builtin_run`.

Given to you: both headers, and the plumbing that isn't where the marks are — `frec_file_path`, `path_join`, `path_is_dir` / `path_is_real_dir`, the `qsort` comparator, the growable `Buffer`, `print_line`, `read_exactly`, `is_executable_file`, `make_absolute`, and the outer loop shapes so you can see what each function receives and returns.

Yours to write, by command:

`hop` — `frec_load` / `frec_save` (the file format), `frec_score` and `frec_better` (the ranking), `frec_record` (bump or insert), the substring search in `hop_freccency_lookup`, `hop_change_dir`, `hop_expand_path`, and the three-attempt logic in `hop_one`.

`reveal` — the `readdir` filter, the printing and recursion in `reveal_list`, and flag/argument parsing.

`peek` — `peek_forward`, `count_non_empty_lines`, the backward chunk scan, the line-index pass for streams, and the per-file loop.

`locate` — the cwd check and the `PATH` walk.

Order I'd suggest: `locate` first (self-contained, no state), then `reveal`, then `hop`, then `peek` last.

These things that will get you marks if you work them:

Write a message...



Sonnet 4.6 Low



```
1  #ifndef BUILTINS_H
2  #define BUILTINS_H
3
4  int builtin_is_builtin(const char *name);
5
6  int builtin_run(int argc, char **argv);
7  void builtin_error(const char *message);
8
9  int reveal_command(int argc, char **argv);
10 int peek_command(int argc, char **argv);
11 int locate_command(int argc, char **argv);
12
13 #endif
```

Order I'd suggest: `locate` first (self-contained, no state), then `reveal`, then `hop`, then `peek` last.

Three things that will cost you marks if you rush them:

`hop_expand_path` is called by `reveal` too. Both commands resolve `~ . . .` identically — only the frequency fallback differs, and that lives in `hop_one`, not here. Writing the rules twice is how the two commands drift apart.

In `peek_reverse_seekable`, the reason `partial` exists: a 4096-byte chunk boundary lands in the middle of a line, so the bytes at the *right* edge of a chunk belong to a line whose start you haven't read yet. Only the first line you complete while scanning right-to-left needs `partial` appended.

For `-r -n`, you can't number lines while reading backwards — line 1 is still line 1. That's why `count_non_empty_lines` runs first: it gives you the total, and you count *down* from it.

Test `peek -r` against `tac` on a file with lines longer than 4096 bytes. That's where the chunk logic breaks if it's wrong, and a small test file will never catch it.



B hop

C

Download



B builtins

C

Download



B hop

H

Download



B builtins

H

Download

Download all

just now

Write a message...



Sonnet 4.6 Low



```

1  #ifndef BUILTINS_H
2  #define BUILTINS_H
3
4  int builtin_is_builtin(const char *name);
5
6  int builtin_run(int argc, char **argv);
7  void builtin_error(const char *message);
8
9  int reveal_command(int argc, char **argv);
10 int peek_command(int argc, char **argv);
11 int locate_command(int argc, char **argv);
12
13 #endif

```

For this part, when proces
sing commands with sequ
ential (;) or background
(&) operators, you should
only execute the first com
mand group and ignore ...

PASTED

Now we do the part C, give me the skeleton and I will fill in the blank functions

Ran 2 commands, read a file

Part C skeleton — one header, one source. All four sections live in `c_exec.c`.

Given to you: the header with the `Redirection` struct, plus `is_runnable`, `join_path`, the `search_in_path` loop shape, `copy_all`, `make_scratch_file`, `close_fd_list`, `redir_init`, `redir_open`, and the outer shape of the pipeline loop so the ordering of pipe → fork → close is visible.

Yours to write:

Section	Blanks
C1	the <code>search_in_path</code> body, <code>exec_resolve</code> , <code>child_exec</code>
C2	<code>open_input</code>
C3	<code>open_output</code> , <code>redir_apply</code> , <code>redir_finish</code>
C4	the child branch, the parent's closing, and the wait loop

Do C1 first and test with `echo hello` and `dosakdaoskdos` before touching redirection. Each section builds on the last.

The parts where the marks actually are:



Write a message...

```
1  #ifndef EXEC_H
2  #define EXEC_H
3
4  #include "a_command.h"
5
6  #define EXEC_PATH_MAX 4096
7
8  typedef struct {
9      int in_fd;
10     int out_fd;
11     int targets[MAX_REDIRECTS];
12     int target_count;
13     int fan_out;
14 } Redirection;
15
16 // C2 and C3
17 int redir_open(Command *cmd, Redirection *redir);
18
19 void redir_apply(Redirection *redir);
20
21 void redir_finish(Redirection *redir);
22
23 // C1 and C4
24 int exec_run_pipeline(Job *job);
25
26 int exec_resolve(const char *name, char *out, int out_size);
27
28 #endif
```


`open_input` and `open_output` face the same problem from opposite ends — a command can list several files, but `stdin` and `stdout` are each a single descriptor. Both solve it with `make_scratch_file`: input concatenates *into* it beforehand, output fans *out* of it afterwards. In `redir_finish`, rewind before every single copy, not once before the loop.

In the pipeline, the parent closing its pipe descriptors immediately after `fork` isn't tidiness — it's correctness. A reader only sees EOF when *every* write end is closed, and the parent holds one until it lets go. Miss it and `%yes | %head -2` hangs forever.

Resolve the command name inside `child_exec`, not in the parent before forking. That's what makes requirement 8 fall out for free: a bad stage prints its error, `_exit` s, and its pipe closes on its own, so the following stages just see empty input and run normally.

Order matters in the child: pipe `dup2` first, then `redir_apply`. An explicit `> out.txt` on the last stage has to win over the pipe wiring.

Two things worth testing beyond the spec examples: `%yes | %head -2` under a timeout, to prove nothing hangs, and `%ls /proc/self/fd` after a few redirections, to prove you aren't leaking descriptors.

C exec

C

Download

C exec

H

Download

Download all

1 minute ago



Write a message...



Sonnet 4.6 Low



```

1  #ifndef EXEC_H
2  #define EXEC_H
3
4  #include "a_command.h"
5
6  #define EXEC_PATH_MAX 4096
7
8  typedef struct {
9      int in_fd;
10     int out_fd;
11     int targets[MAX_REDIRECTS];
12     int target_count;
13     int fan_out;
14 } Redirection;
15
16 // C2 and C3
17 int redir_open(Command *cmd, Redirection *redir);
18
19 void redir_apply(Redirection *redir);
20
21 void redir_finish(Redirection *redir);
22
23 // C1 and C4
24 int exec_run_pipeline(Job *job);
25
26 int exec_resolve(const char *name, char *out, int out_size);
27
28 #endif

```